STEALTH ASSASSIN

Game Design Document

Shadows Never Forgive

Project: Design Patterns

Year: 2026

Team Members:

Onalbaev Aidos • Seisembay Asylbek • Aruzhan Kuat

Built with libGDX | Java

1. Game Summary

Stealth Assassin is a 2D stealth-based RPG (top-down view) built with libGDX. The player chooses one of three unique heroes — Ninja, Shadow, or Rogue — and must eliminate enemies using detection mechanics, stealth movement, and adaptive AI behavior.

Each enemy has its own AI state (Patrol, Chase, Attack, Search) controlled through the State design pattern. The player wins by defeating all enemies; the player loses when their HP reaches zero. The project demonstrates 10 software design patterns: Singleton, Factory Method, Abstract Factory, Builder, Prototype, Object Pool, Decorator, Facade, State, and Observer.

Genre: 2D Stealth-RPG (Top-down)

Core mechanic: Stealth movement, detection avoidance, strategic combat

Win condition: Defeat all enemies on the level

Lose condition: Player HP reaches 0

Platform: Desktop (Windows, macOS, Linux) via LWJGL3

2. Controls

Key / Button	Action
W / A / S / D	Move hero (up, left, down, right)
SPACE	Activate hero's special ability
F	Attack the closest enemy in range (60px)
ENTER	Confirm selection in menus
ESC	Return to main menu / quit game
1 / 2 / 3	Quick hero selection in selection screen
E / H	Choose difficulty (Easy / Hard)

3. Player Stats & Heroes

The player can choose between three heroes, each with a unique playstyle and special ability:

Hero	HP	Speed	Stealth	Damage	Special Ability
Ninja	250	200	0.9	30	Shadow Strike (damage x2 for 5s)
Shadow	150	180	1.0	25	Invisibility (enemies can't see for 5s)
Rogue	200	150	0.7	45	Iron Skin (max HP +50, one-time)

Note: Stealth value reduces enemy detection radius. A higher stealth means enemies see you from a shorter distance.

4. Enemies

There are 5 different enemy types, each with unique stats and behaviors. Their AI is controlled by the State pattern (Patrol → Chase → Attack → Search).

Enemy	HP	Damage	Detection	Score	Special
Guard	50	15	150 px	+10 pts	Basic patrol enemy
Archer	40	20	250 px	+15 pts	Long-range attacks
Knight	120	30	180 px	+25 pts	30% armor reduction
Mage	60	35	280 px	+30 pts	Magical attacks (mana)
Boss	500	60	400 px	+100 pts	Multi-phase, x1.5 speed at low HP

Enemy AI States (State Pattern)

- PatrolState — Default state. Enemy walks back and forth on a route.
- ChaseState — When the player is detected within radius, enemy pursues at 1.5x speed.
- AttackState — When close enough (50px), enemy attacks every 1.2 seconds.
- SearchState — When player is lost, enemy searches for 5 seconds, then returns to PatrolState.

Aggro System: When an enemy takes damage, it switches to ChaseState for 10 seconds and remembers the player's position dynamically.

5. Levels

The game has 2 difficulty levels, generated dynamically by the Abstract Factory pattern (EasyLevelFactory and HardLevelFactory).

Level	Enemy Count	Enemy Composition
Easy	3	2x Guard, 1x Archer
Hard	5	2x Knight, 1x Mage, 1x Archer, 1x Boss

Map Description

- Map size: 1280 x 720 pixels (HD resolution)
- Floor: Procedurally generated stone tile texture (64x64 px tiles)
- Walls: Brick texture surrounding the play area + center obstacles for cover
- Lighting: Dim atmosphere fitting a stealth game

6. Win / Lose Conditions

Victory Screen

Triggered when all enemies on the level are defeated. Shows:

- Green-themed background
- 'JENIS!' (Victory) headline
- Final score with bonus points
- Options: ENTER to return to main menu, ESC to exit

Defeat Screen

Triggered when the hero's HP reaches 0. Shows:

- Red-themed background
- 'JENILIS' (Defeat) headline
- Final score earned
- Options: ENTER to retry, ESC to exit

Scoring System (Observer Pattern)

Score is tracked through the Observer pattern. ScoreObserver listens to game events:

- ENEMY_KILLED event — adds points based on enemy type
- STEALTH_KILL event — bonus +50 points if enemy was undetected

7. Art Style

Style: Programmatic Pixel Art

All graphics are generated procedurally in code using libGDX's Pixmap API — no external image files are required. This makes the build self-contained and easily portable.

Color Palette

- Heroes: Cyan (Ninja), Purple (Shadow), Red (Rogue)
- Enemies: Orange (Guard), Green (Archer), Gray (Knight), Magenta (Mage), Dark Red (Boss)
- UI: Black background, blood-red accents, golden highlights
- Detection radius: Yellow (patrol) → Red (chase)

Visual Effects

- Animated stars on menu screens
- Rotating shurikens in background
- Fog overlay for depth
- Glitch effect on titles
- Pulsing selection indicators
- Red flash when hero takes damage
- Green attack indicator on player attack

8. Design Patterns Used

This project demonstrates 10 GoF (Gang of Four) software design patterns:

№	Pattern	Where Used
1	**Singleton**	StealthAssassinGame, AssetManager, GameEventManager, BulletPool — single instance throughout the game.
2	**Factory Method**	HeroFactory.createHero(), EnemyFactory.createEnemy() — encapsulate object creation logic.
3	**Abstract Factory**	AbstractGameFactory + EasyLevelFactory + HardLevelFactory — create families of related enemies for difficulty levels.
4	**Builder**	LevelBuilder — step-by-step construction of complex Level objects (name, map, enemies, fog, night mode).
5	**Prototype**	Enemy.clone() — clone enemies for fast spawning without rebuilding.
6	**Object Pool**	BulletPool — reuse Bullet objects to avoid garbage collection (performance).
7	**Decorator**	HeroDecorator + StealthBoost / SpeedBoost / DamageBoost — dynamic ability enhancement.

8	**Facade**	GameFacade — simplifies starting a new game by hiding factories, observers and builder behind one API.
9	**State**	EnemyState + Patrol/Chase/Attack/Search — enemy AI behavior changes based on its state.
10	**Observer**	GameEventManager + ScoreObserver — decoupled event system (e.g., enemy killed → score updates).

9. Team & Responsibilities

Member	Role	Responsibilities
Onalbaev Aidos	Project LeadLead Programmer	ClickUp board, deadlines, code reviews, core architecture (StealthAssassinGame, GameFacade), all Screens, UI design.
Seisembay Asylbek	Programmer	All Hero classes (Ninja, Shadow, Rogue), all Enemy classes (Guard, Archer, Knight, Mage, Boss), Factory pattern implementations.
Aruzhan Kuat	Programmer & Designer	State pattern (4 states), Observer pattern, Decorator pattern, Object Pool, Vector2D utility, TextureGenerator (sprite art).

10. SOLID Principles in Code

Our project follows all five SOLID principles. These principles make the code easier to extend, debug, and maintain.

Letter	Principle	Definition	Where Applied in Our Code
S	**Single Responsibility**	Each class has only one reason to change. One class = one job.	InputManager handles only input. DetectionManager handles only stealth detection. AssetManager handles only resources. ScoreObserver handles only score tracking.
O	**Open / Closed**	Classes should be open for extension but closed for modification.	To add a new enemy (e.g., Wizard), we extend the abstract Enemy class without changing existing code. Same for Hero — adding a new hero only requires extending Hero.
L	**Liskov Substitution**	Subclasses must work wherever the parent class works.	Hero hero = new Ninja(); — works correctly. Replacing it with new Shadow() or new Rogue() also works without breaking the game. All subclasses honor the Hero contract.
I	**Interface Segregation**	Don't force classes to implement methods they don't need.	EnemyState interface has only 2 methods: handle() and getStateName(). GameEventListener has only one: onEvent(). No bloated interfaces — each is small and focused.
D	**Dependency Inversion**	Depend on abstractions, not concrete classes.	GameFacade depends on the abstract Hero / Enemy classes (not on Ninja/Guard directly). Enemy uses EnemyState interface, not specific PatrolState. Easy to swap implementations.

Concrete Code Examples

Below are real examples from our codebase showing each SOLID principle in action:

Single Responsibility (S):

- InputManager.handleInput() — only listens to keyboard, nothing else
- DetectionManager.checkDetection() — only checks if hero is visible to enemies
- ScoreObserver.onEvent() — only updates score on game events

Open / Closed (O):

- New enemy types (Mage, Boss) added without modifying Enemy base class
- New decorators (StealthBoost, SpeedBoost) extend HeroDecorator without changing Hero
- New states (e.g., FleeState) can be added without changing Enemy logic

Liskov Substitution (L):

- Hero hero = heroFactory.createHero(NINJA, x, y) — same code works for any hero type
- List<Enemy> enemies — can hold Guards, Mages, Bosses, all behave correctly as Enemy
- Boss.takeDamage() respects parent contract while adding phase logic

Interface Segregation (I):

- EnemyState — only handle() and getStateName() (no bloat)
- GameEventListener — only onEvent() method
- Cloneable (used by Enemy) — only clone() method

Dependency Inversion (D):

- GameFacade.startNewGame() works with abstract AbstractGameFactory, not Easy/Hard directly
- Enemy.currentState: EnemyState (interface), not concrete state class
- HeroDecorator.decoratedHero: Hero (abstract), not Ninja/Shadow/Rogue

11. Project Milestones

Milestone	What's Delivered	Description
M1: Design	GDD + Diagrams	Game Design Document, Class Diagram, Game Flow Diagram, Level Sketch.
M2: Prototype	Core mechanic	Hero movement, enemy patrol, basic detection — playable but unpolished.
M3: Alpha	All features	All 10 patterns, 3 heroes, 5 enemies, 2 levels, full HUD.
M4: Beta	Bug fixes	Stealth mechanic polished, AI behavior balanced, UI improved.
M5: Final	Polished build	Cinematic menus, all visual effects, runnable JAR, ready to submit.

— End of Document —